Generalización de Código de Automatización en Desarrollo de Software con IA

Tema 2. Modelos generativos específicos para programación

Índice

Esquema

Ideas clave

2.1. Introducción y objetivos

2.2. Introducción a los modelos generativos en programación

2.3. GAN en generación de código

2.4. Transformer models para secuencias de código

2.5. Entrenamiento y sintonización de modelos generativos

2.6. Implementación práctica de modelos en proyectos de desarrollo

2.7. Referencias bibliográficas

A fondo

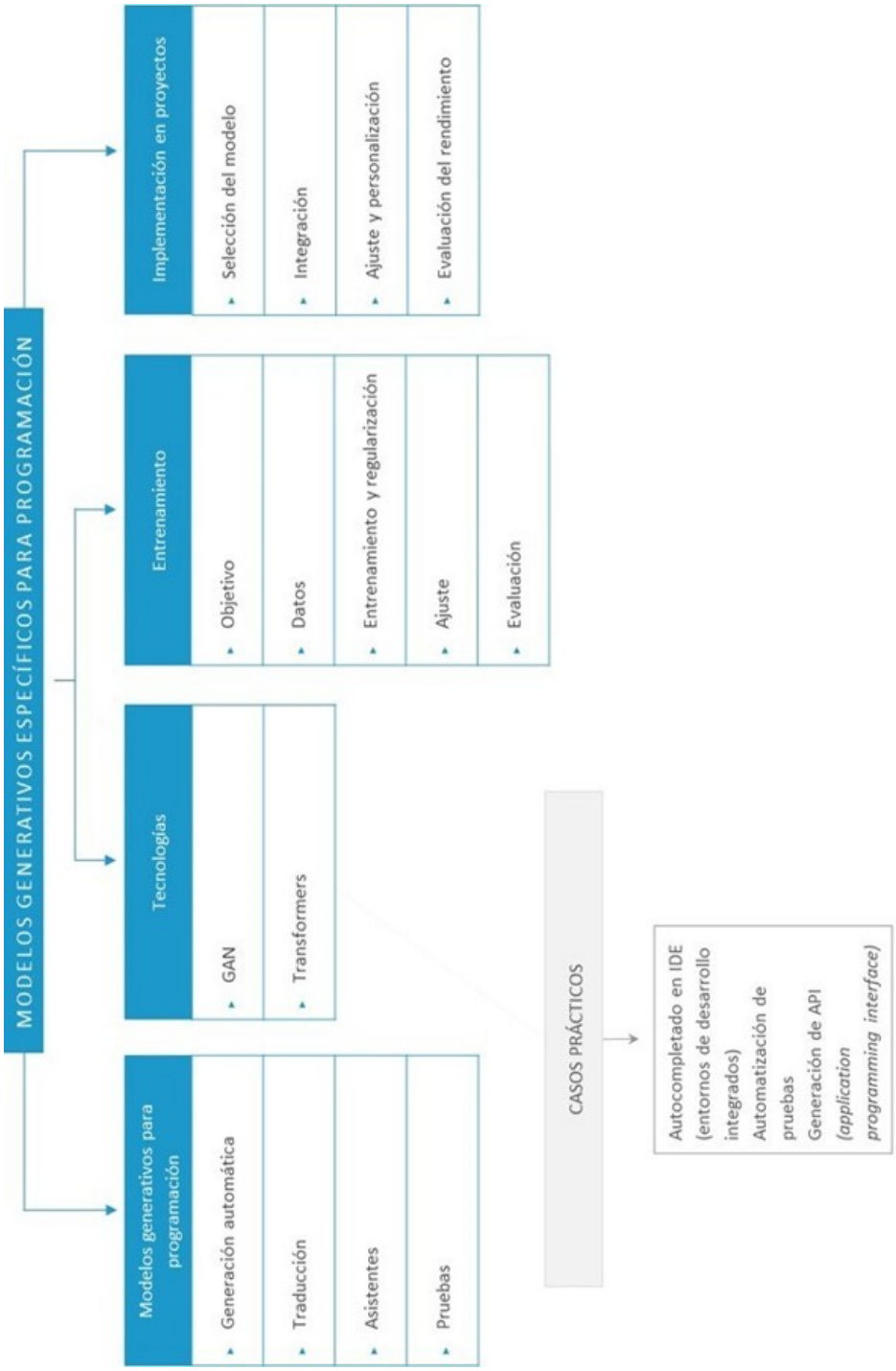
OpenAI Codex: IA que escribe software

Exploring the limits of transfer learning with a unified text-to-text transformer

Microsoft CodeBERT

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

2.1. Introducción y objetivos

Los **modelos generativos específicos para programación** representan una de las mayores innovaciones recientes dentro del campo de la inteligencia artificial (IA) aplicada al desarrollo de *software.* Estos modelos utilizan redes neuronales avanzadas, como las **redes generativas adversarias** (GAN) y los transformadores, para generar automáticamente un código funcional a partir de descripciones en lenguaje natural, la **traducción de código entre diferentes lenguajes** y la realización de tareas complejas, como la generación de pruebas automáticas o la optimización de código.

La introducción de modelos generativos en el desarrollo de *software* redujo significativamente el tiempo y los esfuerzos necesarios para **escribir, optimizar y probar código.** Herramientas como Codex y GitHub Copilot demostraron cómo la IA puede proporcionar **asistencia en tiempo real** a los desarrolladores, lo que mejora la eficiencia y la calidad del código. Además, la capacidad de estos modelos para aprender de grandes volúmenes de datos significa que pueden **adaptarse a diversas aplicaciones,** desde la creación de funciones simples hasta la generación de estructuras complejas en sistemas grandes.

Los modelos transformadores como GPT-3 y sus derivados fueron fundamentales para mejorar la capacidad de los desarrolladores en tareas como el **autocompletado de código,** la traducción automática entre lenguajes de programación y la corrección automática. Estos avances han sido impulsados por grandes cantidades de **datos de código fuente disponibles** en plataformas como GitHub, lo que permitió que los modelos generativos no solo aprendan patrones de lenguaje, sino también las **mejores prácticas de codificación.**

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Los objetivos de este tema son:

- Comprender los fundamentos de los modelos generativos aplicados a la programación.
- Explorar las aplicaciones prácticas de los modelos generativos.
- Conocer los desafíos y limitaciones actuales.
- Desarrollar habilidades para implementar modelos generativos.

© Universidad Internacional de La Rioja (UNIR)

2.2. Introducción a los modelos generativos en programación

Un modelo generativo es un tipo de algoritmo de aprendizaje automático que **aprende a modelar la distribución de datos** a partir de un conjunto de entrenamiento. A diferencia de los modelos discriminativos (que se centran en clasificar o predecir categorías), los modelos generativos pueden **crear nuevos datos** similares a los del conjunto de entrenamiento.

En programación, esto significa que pueden aprender de bases de código existentes y generar nuevas soluciones de *software,* optimizaciones o incluso traducir código entre diferentes lenguajes.

El salto de los modelos generativos al ámbito de la programación comenzó con la **adaptación de modelos** basados en redes neuronales recurrentes (RNN) y, más tarde, con el auge de las redes neuronales tipo *transformer.* Esto permitió a los modelos comprender no solo la **estructura del lenguaje natural,** sino también la **sintaxis y semántica** de los lenguajes de programación.

Aplicaciones de modelos generativos en programación

- **Generación automática de código.** Los modelos como Codex (desarrollado por OpenAI) son capaces de generar código a partir de descripciones en lenguaje natural. Esto permite que los desarrolladores escriban una descripción de lo que quieren que un bloque de código haga y que el modelo genere el código correspondiente en el lenguaje de programación deseado.

- Por ejemplo, dado el enunciado «escribir una función en Python que ordene una lista de enteros mediante el algoritmo de ordenación por inserción», un modelo como Codex puede producir el código completo que implementa dicho algoritmo.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Traducción entre lenguajes de programación.** Los modelos generativos también pueden ser entrenados para traducir código entre diferentes lenguajes de programación. Esto es útil en casos de migración de *software* o portabilidad entre plataformas.

  - Por ejemplo, un modelo puede traducir código Java a Python, por lo que puede mantener la lógica intacta y adaptar la sintaxis, de acuerdo con las convenciones de cada lenguaje.

- **Asistentes inteligentes para desarrollo.** Herramientas como GitHub Copilot, basadas en modelos generativos, proporcionan asistencia en tiempo real durante el desarrollo. Estas herramientas sugieren código, mientras que el programador trabaja, por lo que se basan en el contexto del código ya escrito.

GitHub Copilot aprendió de una vasta cantidad de código abierto, lo que le permite ofrecer sugerencias precisas en una variedad de lenguajes y contextos.

- **Generación automática de pruebas.** Algunos modelos pueden generar pruebas automáticas basadas en el código ya existente. Esto es crucial para mejorar la calidad del *software* y garantizar que todas las funcionalidades estén debidamente probadas.

  - Por ejemplo, un modelo generativo puede crear automáticamente pruebas unitarias que verifiquen el comportamiento de una función en varios escenarios, para asegurar así su correcto funcionamiento.

© Universidad Internacional de La Rioja (UNIR)

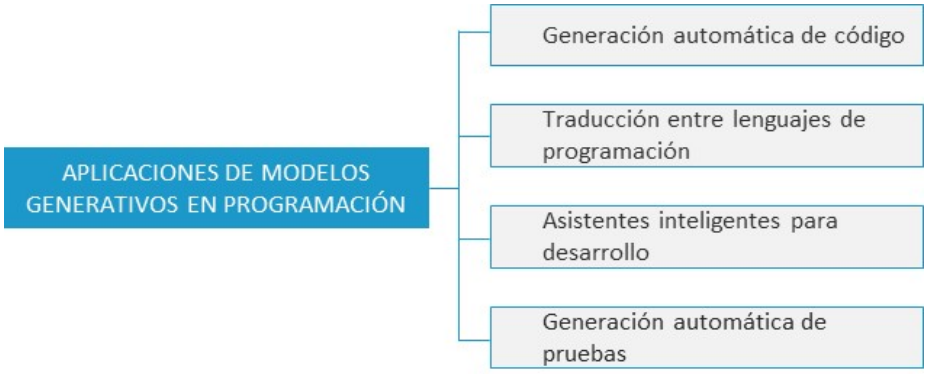


Figura 1. Aplicaciones de modelos generativos en programación. Fuente: elaboración propia.

Tecnologías relevantes en modelos generativos

Existen varios tipos de arquitecturas de modelos generativos que fueron claves en el avance de la generación de código:



Figura 2. Tecnologías relevantes en modelos generativos. Fuente: elaboración propia.

- ***Transformers.*** Los modelos basados en la arquitectura *transformer,* como GPT-3 o Codex, demostraron ser extremadamente eficientes en la generación de secuencias, lo que los hace ideales para la programación, ya que el código es esencialmente una secuencia de instrucciones.

  - Por ejemplo, GPT-3 puede ser utilizado para generar funciones completas de JavaScript y su capacidad de comprensión contextual le permite manejar lenguajes de programación de manera efectiva.

  - La Figura 3 muestra el funcionamiento de la arquitectura *transformer* para generación de código.

© Universidad Internacional de La Rioja (UNIR)

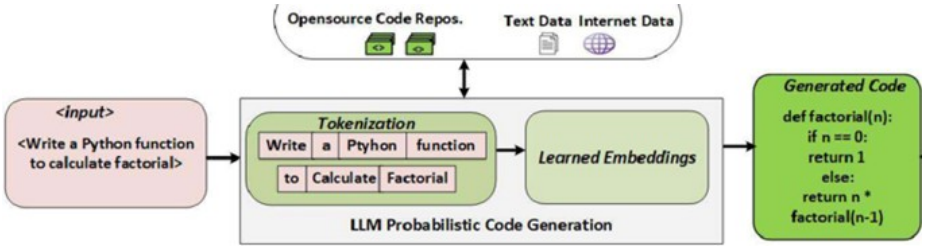


Figura 3. Arquitectura *transformer* para generación de código: Fuente: Mohsin, 2024.

- **GAN.** Aunque las GAN son más comunes en la generación de imágenes, comenzaron a explorarse para la generación de código en dominios específicos, especialmente en la optimización de código y en la creación de estructuras complejas.

- ***Autoencoders* variacionales (VAE).** Como son utilizados principalmente para la optimización y compresión de código, los VAE pueden aprender representaciones latentes que ayudan a mejorar la eficiencia y la estructura del código generado.

Desafíos y futuro de los modelos generativos en programación

Uno de los mayores desafíos con los modelos generativos en programación es garantizar que el código generado sea eficiente, seguro y libre de errores.

A pesar de que los modelos avanzaron considerablemente, aún es necesario realizar **pruebas exhaustivas y validaciones** para asegurarse de que el código generado sea adecuado para su uso en producción.

Otro desafío es la **ética del código generado.** Modelos como Codex fueron entrenados en grandes cantidades de código de repositorios públicos, lo que plantea preguntas sobre los **derechos de autor y la propiedad intelectual** del código generado.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

A pesar de estos desafíos, el futuro de los modelos generativos en programación es prometedor. Por tanto, se espera que **continúen mejorando en precisión** y que jueguen un papel crucial en la **automatización de tareas de desarrollo,** para permitirles a los programadores centrarse en problemas más complejos y creativos.

© Universidad Internacional de La Rioja (UNIR)

2.3. GAN en generación de código

Las GAN son una clase de modelos generativos que se utilizan principalmente en la **generación de imágenes, audio y vídeos,** pero comenzaron a explorarse en el ámbito de la generación de código. Aunque las GAN no son tan comúnmente utilizadas en la **generación de secuencias,** como los modelos basados en la arquitectura *transformer,* su capacidad para generar datos sintéticos de alta calidad las hace una herramienta atractiva en la **creación y optimización de código.**

Las GAN consisten en **dos redes neuronales** que compiten entre sí en un proceso de entrenamiento adversarial:

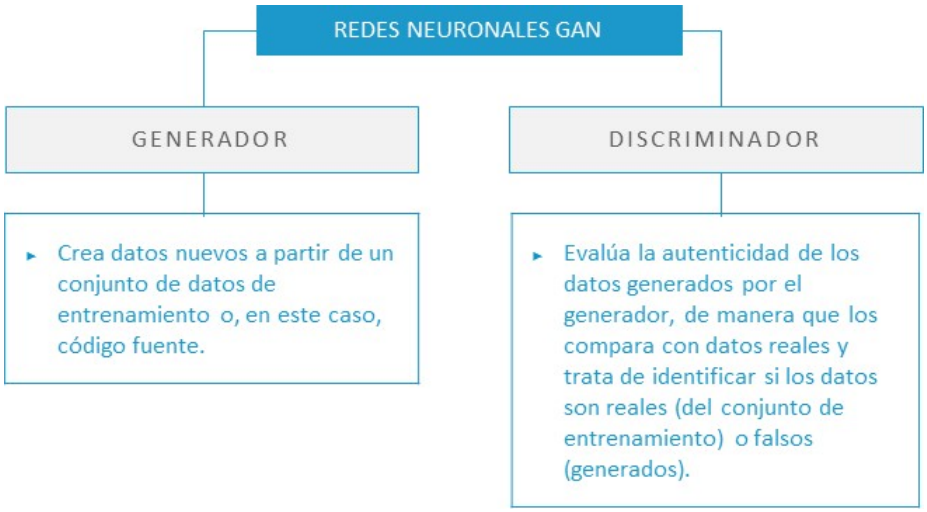


Figura 4. Redes neuronales GAN. Fuente: elaboración propia.

Durante el proceso, el generador mejora sus capacidades para engañar al discriminador, mientras que el discriminador mejora en detectar las falsificaciones. Este ciclo adversarial lleva a que el generador aprenda a crear datos que imiten con mayor precisión a los datos reales.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Las GAN ofrecen una perspectiva única en la generación de código, por lo que se enfocan principalmente en la **optimización** y en la **creación de código sintético** para mejorar la productividad de los desarrolladores. Aunque su aplicación en programación aún está en una fase exploratoria, los avances en este campo podrían llevar a **nuevas formas de escribir, mejorar y mantener código** en el futuro cercano.

Optimización del código

Uno de los usos más interesantes de las GAN en programación es la **optimización de código.** En lugar de generar código desde cero, las GAN pueden **aprender patrones de eficiencia** a partir de grandes conjuntos de código optimizado y aplicar estos patrones para **refactorizar o mejorar código existente.**

> Por ejemplo, imagina un generador que crea varias versiones optimizadas de un bloque de código en C++, mientras que el discriminador evalúa si las versiones generadas son más rápidas o utilizan menos memoria que el código original. El generador puede aprender a producir versiones del código con mejor rendimiento sin alterar su funcionalidad.

En lugar de depender de reglas específicas para la optimización, como lo harían los compiladores tradicionales, las GAN pueden aprender directamente de los **datos históricos** de código eficiente, lo que podría **permitir innovaciones** en la forma en que se optimizan programas.

Generación de código sintético

Otro campo prometedor es la generación de **código sintético.** En muchos casos, es necesario contar con ejemplos extensos de código para **entrenar modelos de aprendizaje automático,** pero estos ejemplos pueden no estar disponibles debido a **restricciones de derechos de autor o confidencialidad.**

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Al generar código sintético, las GAN pueden crear soluciones innovadoras que no solo imiten el código de entrenamiento, sino que también combinen elementos de diferentes ejemplos para generar nuevos patrones.

Las GAN pueden generar **ejemplos de código sintético** que imitan las características del código real, lo que les permite a los investigadores y desarrolladores contar con **datos adicionales** sin infringir ninguna propiedad intelectual.

> Por ejemplo, una GAN entrenada en un conjunto de código fuente de aplicaciones Java podría generar ejemplos sintéticos que sigan las mismas estructuras y patrones, lo que ayudaría a entrenar modelos de aprendizaje profundo sin necesidad de utilizar código propietario.

Corrección y refactorización automática

Las GAN también pueden ser utilizadas para **mejorar el código existente mediante refactorización automática,** lo que mantiene su funcionalidad, pero mejora su estructura o legibilidad. Aquí, el generador se encargaría de **producir una versión mejorada del código,** mientras que el discriminador evaluaría si dicha versión es más **legible o eficiente que el código original.**

> Por ejemplo, supón que una GAN está entrenada en un gran conjunto de código Python bien estructurado. Puede aprender a identificar y corregir patrones de código mal estructurados o redundantes, para generar una versión más limpia del mismo bloque de código.

© Universidad Internacional de La Rioja (UNIR)

Desafíos en la aplicación de GAN en programación

- **Secuencialidad del código.** A diferencia de las imágenes o el audio, el código tiene una estructura secuencial rígida, lo que presenta un reto para las GAN, que tradicionalmente fueron más efectivas en datos de formato continuo o espacial (como imágenes).

- **Evaluación de la corrección.** Mientras que en imágenes la evaluación es más subjetiva (por ejemplo, si una imagen es realista o no), en programación es crítico que el código generado sea funcional y correcto. Por lo tanto, es necesario asegurar que el discriminador no solo evalúe si el código generado se parece al código real, sino también si es ejecutable y cumple con los requerimientos.

- **Limitaciones en la creatividad.** Aunque las GAN pueden generar código sintético o mejorar el código existente, su capacidad para generar soluciones completamente nuevas o creativas está limitada por los datos con los que se entrenan. Esto plantea un desafío para generar innovaciones reales en lugar de soluciones derivadas.

GANCoder

El modelo GANCoder es un **enfoque innovador** que combina las GAN con técnicas de generación de código para crear un modelo que pueda **generar y optimizar automáticamente** fragmentos de código. Este método se basa en la **interacción de dos redes neuronales:** un generador que crea fragmentos de código y un discriminador que evalúa la calidad de ese código.

GANCoder tiene como objetivo generar código que no solo sea funcional, sino también optimizado y coherente con las mejores prácticas de programación.

© Universidad Internacional de La Rioja (UNIR)

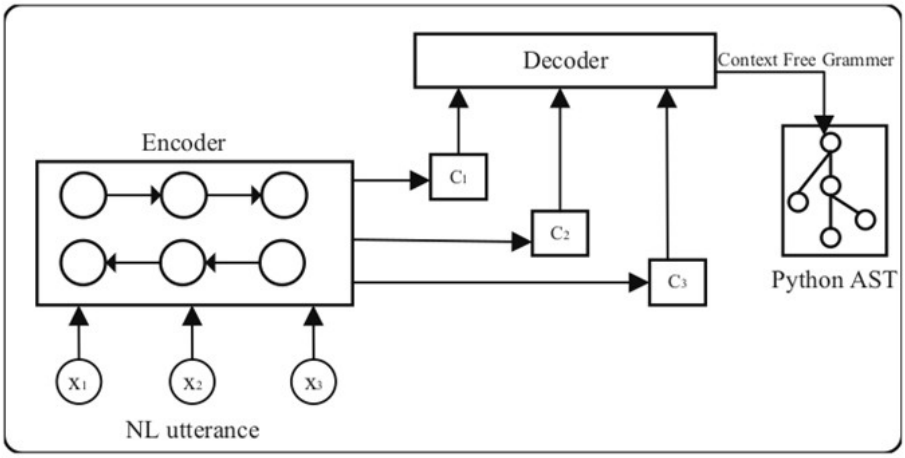


Figura 5. Arquitectura empelada en el modelo GANCoder. Fuente: Zhu et al., 2019.

A medida que las GAN evolucionan y se integran con otras arquitecturas más adecuadas para la **secuencialidad del código** (como las basadas en *transformers*), su capacidad para generar y optimizar código mejorará. Se espera que en el futuro puedan no solo **optimizar el código existente** o generar ejemplos sintéticos, sino también **combinar diferentes patrones de programación** para ofrecer soluciones más innovadoras y eficientes.

© Universidad Internacional de La Rioja (UNIR)

2.4. Transformer models para secuencias de código

Los modelos *transformer* revolucionaron el **procesamiento de lenguaje natural** (PLN) y también están transformando la generación y el análisis de código. Originalmente introducidos por Vaswani et al. (2017), los *transformers* se basan en un **mecanismo de atención** que les permite manejar grandes secuencias de datos de manera más eficiente que los modelos tradicionales, como las RNN o las redes de memoria a largo plazo (LSTM). En el contexto de la programación, estos modelos son extremadamente valiosos, ya que el código fuente es, en esencia, una **secuencia de instrucciones estructuradas.**

La principal ventaja de los *transformers* es que permiten procesar contextos largos, algo especialmente útil en programación, donde la dependencia entre diferentes partes del código puede estar separada por múltiples líneas o archivos de código.

Los *transformers* son modelos de aprendizaje profundo que utilizan **mecanismos de atención autoreguladora** para procesar secuencias enteras de datos simultáneamente, en lugar de hacerlo paso a paso como en los enfoques secuenciales tradicionales. Además, para no procesar cada palabra o *token* (que, en este caso, serían líneas de código) en una secuencia de manera ordenada, los *transformers* pueden **identificar relaciones entre las partes** de una secuencia independientemente de su distancia.

© Universidad Internacional de La Rioja (UNIR)

- **Atención y contexto global**. En la generación de código, es crucial entender el contexto. Un modelo *transformer* puede prestar atención tanto a las dependencias locales como a las globales dentro de un archivo de código, o incluso entre archivos relacionados. Esto le permite realizar tareas como autocompletar código, detectar errores o generar funciones completas a partir de descripciones.

- **Manejo de dependencias complejas.** En los lenguajes de programación, una variable declarada al inicio puede ser utilizada mucho más adelante. El mecanismo de atención de los *transformers* les permite rastrear estas dependencias sin verse afectados por la longitud de la secuencia. Este aspecto es vital para que el código generado sea coherente y funcional.

  - Por ejemplo, supón que en una función en Python se declara una lista de números y, posteriormente, se aplica una operación a todos los elementos de esa lista. Un *transformer* puede comprender la relación entre la declaración y el uso, sin perder el rastro de la variable a lo largo del código.

- **multitarea**. Los *transformers* pueden entrenarse en múltiples tareas relacionadas con el código, como traducción de lenguajes de programación, detección de errores, optimización de código, o generación de documentación automática. Esto los convierte en herramientas altamente versátiles para los desarrolladores.

Aplicaciones de los modelos transformer en programación

- **Generación automática de código.** Los modelos como Codex, basados en la arquitectura GPT *(generative pre-trained transformer)* (que es un *transformer*), permitieron la generación automática de código a partir de instrucciones en lenguaje natural. Los usuarios pueden describir una función o tarea y el modelo genera el código correspondiente.

  - Por ejemplo, la instrucción «escribir una función en Python que devuelva los números primos entre uno y cien» resultaría en la generación automática de la función correspondiente en Python.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Autocompletado inteligente.** Herramientas como GitHub Copilot (también basado en un *transformer*) ofrecen sugerencias automáticas mientras se escribe código. Utilizan el contexto de lo que el programador ya escribió para predecir las siguientes líneas, lo que mejora significativamente la productividad.

  - GitHub Copilot puede autocompletar no solo pequeñas frases o expresiones de código, sino también estructuras completas como bucles, funciones y clases, por lo que se ajusta al estilo de programación del usuario.

  - La Figura 6 muestra cómo GitHub Copilot, integrado en Visual Studio, es capaz de sugerir el código de una función a partir de la cabecera de esta.



Figura 6. Ejemplo de autocompletado inteligente con GitHub Copilot. Fuente: Visual Studio Code, s. f.

- **Traducción entre lenguajes de programación.** Los *transformers* también se pueden entrenar para traducir código entre diferentes lenguajes, algo muy valioso para la migración de *software.* Al igual que con la traducción de lenguajes humanos, el modelo entiende la estructura y las reglas sintácticas de un lenguaje y las traduce a otro.

  - Por ejemplo, un *transformer* puede convertir un bloque de código en C++ a su equivalente en Python, por lo que mantiene la funcionalidad, pero se ajusta a las convenciones sintácticas de cada lenguaje.

© Universidad Internacional de La Rioja (UNIR)

- **Detección de errores y depuración.** Los *transformers* pueden ser entrenados para detectar patrones de errores comunes en el código, como errores de sintaxis o lógicos, y sugerir correcciones automáticas. Esto es de gran ayuda en ambientes de desarrollo de *software* donde se necesita mantener altos estándares de calidad.

  - Por ejemplo, un *transformer* entrenado en un gran conjunto de datos de código JavaScript puede detectar patrones comunes que conducen a errores de referencia de variables y sugerir soluciones en tiempo real.

- **Optimización de código.** Algunos modelos *transformer* pueden ser entrenados para generar código más eficiente. Esto se logra al alimentar al modelo con ejemplos de código optimizado, lo que le permite aprender a sugerir mejores formas de implementar una solución.

  - Por ejemplo, un *transformer* puede sugerir una mejor implementación de un algoritmo de ordenamiento para una lista larga en Python, de manera que cambia un algoritmo de ordenación ineficiente por uno óptimo.

Ejemplos de modelos basados en transformers para código

- **GPT-3 y Codex.** GPT-3 es una implementación avanzada de *transformer* que, entrenada en grandes corpus de datos de texto y código, puede realizar tareas de generación de código a partir de lenguaje natural. Codex, una extensión de GPT-3, está entrenado específicamente en código fuente, lo que permite resultados más precisos en la generación de código.

- **CodeBERT.** Basado en la arquitectura de BERT *(bidirectional encoder representations from transformers)*, es un modelo diseñado específicamente para tareas de programación. CodeBERT puede realizar tareas como la clasificación de código, la generación de funciones y la comprensión semántica del código, lo que lo hace ideal para tareas como la búsqueda de fragmentos de código relevantes o la depuración automática.

© Universidad Internacional de La Rioja (UNIR)

- **GraphCodeBERT.** Una variante que extiende la funcionalidad de CodeBERT al integrar representaciones gráficas del código, lo que permite un análisis más profundo de la estructura lógica de los programas.

Futuro de los transformers en la generación de código

FUTURO DE LOS *TRANSFORMERS*	
GENERACIÓN DE CÓDIGO	**Entrenamiento en multilenguaje.** Los *transformers* pueden entrenarse en múltiples lenguajes de programación simultáneamente, lo que les permite reconocer y generar código en diferentes lenguajes con relativa facilidad. Por ejemplo, un modelo entrenado en Python, JavaScript y C++ puede identificar similitudes estructurales y ofrecer traducciones entre estos.
	Adaptación continua. Al estar expuestos a grandes cantidades de código de diferentes repositorios, los *transformers* continúan aprendiendo de las mejores prácticas de desarrollo de código, lo que permite que las sugerencias generadas sean cada vez más precisas y acordes con los estándares de la industria.

Tabla 1. Futuro de los *transformers* en la generación de código. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

2.5. Entrenamiento y sintonización de modelos generativos

El entrenamiento y la sintonización de modelos generativos son procesos fundamentales para garantizar que estos modelos puedan **generar resultados de alta calidad,** ya sea texto, imágenes, música o código. En el contexto de la generación de código, estos modelos requieren un **entrenamiento cuidadoso** en grandes cantidades de datos de programación para **comprender patrones, sintaxis, estructuras lógicas y dependencias de código.** Además, la sintonización (o ajuste fino) permite personalizar los modelos para tareas específicas o dominios particulares de programación.

Entrenamiento de modelos generativos

El entrenamiento de un modelo generativo consiste en enseñarle al modelo a generar nuevas muestras (en este caso, código) que se asemejen a las muestras reales en las que fue entrenado.

Este proceso incluye varios pasos clave:

- **Definición del objetivo.** El objetivo del modelo generativo es aprender la distribución subyacente de los datos. En el caso de la generación de código, esto significa que el modelo debe aprender las reglas y patrones de los lenguajes de programación para generar código sintácticamente correcto y funcional. El tipo de tareas generativas puede incluir:

  - Generación de código a partir de lenguaje natural (por ejemplo, «escribe una función en Python para calcular el factorial»).

  - Autocompletado de código a medida que el programador escribe.

© Universidad Internacional de La Rioja (UNIR)

- Traducción de código entre lenguajes (por ejemplo, convertir código de Python a JavaScript).

- **Conjunto de datos para entrenamiento.** El éxito del entrenamiento depende, en gran medida, de la calidad y la cantidad de los datos de entrenamiento. Para los modelos generativos de código, se utilizan conjuntos de datos que contienen grandes colecciones de código fuente extraídos de repositorios públicos como GitHub, Stack Overflow y otras fuentes. Ejemplos de conjuntos de datos:

  - CodeSearchNet es un conjunto de datos que contiene millones de funciones de código en varios lenguajes (Python, Java, JavaScript, etc.), por lo que es utilizado para entrenar modelos de comprensión de código.

  - GitHub Repositories tiene datos extraídos de repositorios públicos de código abierto en GitHub para entrenar modelos en distintos lenguajes y estilos de programación.

- **Preprocesamiento de datos.** Antes de entrenar el modelo, los datos de código necesitan ser preprocesados. Este paso es esencial para convertir el código fuente en una representación que el modelo pueda entender y procesar:

  - *Tokenización:* el código se divide en *tokens,* que son las unidades mínimas de significado (por ejemplo, palabras clave como `if` , `else` , variables, operadores, etc.). Esto es similar a la *tokenización* en el procesamiento de lenguaje natural.

  - Eliminación de comentarios y espacios en blanco: a menudo, se eliminan comentarios o espacios innecesarios para centrarse en el código funcional.

  - Normalización: se puede aplicar normalización para estandarizar estilos de codificación o hacer que el modelo sea más flexible ante variaciones de estilo entre programadores.

- **Entrenamiento con aprendizaje supervisado.** La mayoría de los modelos generativos de código utilizan un enfoque de aprendizaje supervisado, donde el modelo aprende a predecir la siguiente secuencia de código, dado un contexto.

© Universidad Internacional de La Rioja (UNIR)

- Función de pérdida: durante el entrenamiento, se utiliza una función de pérdida (como la pérdida de entropía cruzada) que mide la diferencia entre el código generado por el modelo y el código correcto en los datos de entrenamiento.
- Optimización: el proceso de optimización (generalmente, mediante descenso de gradiente) ajusta los parámetros del modelo para minimizar la función de pérdida.

- **Regularización.** Dado que el código puede tener una estructura altamente repetitiva y compleja, es crucial aplicar técnicas de regularización como la eliminación de ruido o la desactivación de nodos *(dropout)* para evitar que el modelo se ajuste demasiado a patrones específicos, lo que resulta en un sobreajuste *(overfitting)*.

Sintonización fina de modelos generativos

La sintonización fina (o ajuste fino) es el proceso mediante el cual un modelo preentrenado en una gran cantidad de datos generales se ajusta para una tarea o dominio específico. Esto permite que el modelo sea más preciso en tareas concretas sin la necesidad de entrenarlo desde cero.

Por ejemplo, en el desarrollo de GitHub Copilot, los ingenieros realizaron una sintonización fina sobre un modelo GPT para que este se especializara en la generación de código en varios lenguajes. De esta manera, Copilot puede predecir automáticamente el siguiente bloque de código que el usuario necesita escribir, ya que está basado en el contexto actual.

- **Uso de modelos preentrenados.** Los modelos como GPT-3, Codex o CodeBERT son preentrenados en grandes cantidades de datos, pero se pueden sintonizar para tareas más específicas, como la generación de código en un dominio particular. Por ejemplo, generar código para una API *(application programming interface)* específica o un entorno particular.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- ***Transfer learning.*** La sintonización fina aprovecha el aprendizaje transferido, lo que significa que se parte de un modelo preentrenado con un conocimiento general y se ajusta para tareas más especializadas. En el caso de la programación, puede ser:

  - Generación de código en un lenguaje específico (por ejemplo, enfocarse solo en Python o Java).

  - Tareas específicas como generación automática de pruebas unitarias, refactorización de código o análisis de seguridad en código fuente.

- **Proceso de sintonización:**

  - Entrenamiento adicional: se entrena el modelo con un nuevo conjunto de datos que es relevante para la tarea específica. Esto requiere menos datos y menos tiempo de entrenamiento que el entrenamiento desde cero, ya que el modelo ya ha aprendido patrones generales.

  - Ajuste de hiperparámetros: se ajustan los hiperparámetros del modelo, como la tasa de aprendizaje o el número de capas entrenables, para maximizar el rendimiento en la nueva tarea.

  - Validación y evaluación: después de la sintonización, el modelo se evalúa en datos de prueba para asegurarse de que se produzcan resultados precisos y consistentes para la tarea específica.

- **Ejemplos prácticos de sintonización:**

  - Traducción entre lenguajes específicos: un modelo preentrenado en código general puede ser sintonizado para traducir solo entre dos lenguajes, como C++ a Python, lo que mejora su precisión en estos lenguajes.

  - Generación de código para API: un modelo preentrenado puede ajustarse para generar automáticamente llamadas a API de servicios específicos, por lo que puede aprender sus convenciones y reglas.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Para desarrollar una herramienta que genere automáticamente pruebas unitarias, se puede sintonizar un modelo preentrenado en generación de código para que se enfoque específicamente en generar funciones de prueba, como aquellas que validan los resultados de una función principal.

Consideraciones en el entrenamiento y sintonización

- **Datos sesgados.** Uno de los problemas al entrenar y sintonizar modelos generativos es que los conjuntos de datos pueden estar sesgados. En programación, esto puede llevar a que el modelo prefiera ciertos patrones de código, estilos o soluciones, lo que puede no ser deseable en todos los casos.
  - Por ejemplo, un modelo entrenado en código C++ de baja calidad podría generar código subóptimo.
- **Desafíos computacionales.** El entrenamiento y la sintonización de modelos generativos requieren grandes recursos computacionales, especialmente cuando se entrenan con grandes cantidades de código. Esto es debido a la alta dimensionalidad de los datos y a la necesidad de ajustar un gran número de parámetros.
- **Evaluación del modelo.** La evaluación de los modelos generativos de código es compleja. No solo es necesario evaluar la precisión (si el código generado es correcto), sino también aspectos como la eficiencia (si el código es óptimo), la legibilidad (si sigue buenas prácticas de programación) y la coherencia (si el código es consistente a lo largo de diferentes ejecuciones).

© Universidad Internacional de La Rioja (UNIR)

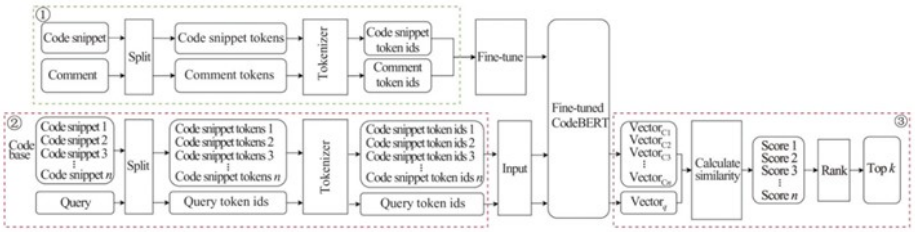


Figura 7. *Fine-tunning* de un modelo CodeBERT. Fuente: Huan y Qinying, 2023.

La Figura 7 muestra el **proceso de *fine-tunning*** de un modelo CodeBERT para búsqueda de código, de forma que se puedan encontrar fragmentos de código similares. Para realizar el ajuste, se emplean pares ya existentes de comentario-código para que el modelo **aprenda a identificar comentarios y códigos similares.**

© Universidad Internacional de La Rioja (UNIR)

2.6. Implementación práctica de modelos en proyectos de desarrollo

La implementación práctica de modelos generativos en proyectos de desarrollo de *software* está transformando la forma en que los **desarrolladores crean, prueban y mantienen *softwares.*** Desde la generación de código automatizado hasta la optimización de tareas repetitivas, los modelos generativos están integrándose de manera fluida en los **flujos de trabajo de desarrollo.** A continuación, se detallan los aspectos clave y las fases para implementar estos modelos en un entorno práctico.

Selección del modelo generativo

El primer paso para implementar un modelo generativo es **seleccionar el modelo adecuado** según las necesidades del proyecto. Existen varios tipos de modelos generativos aplicables al desarrollo de *software,* como:

- GPT-3 y Codex. Estos modelos son especializados en generación de texto y código, por lo que son útiles para el autocompletado, la generación de *scripts* y la traducción entre lenguajes de programación.
- CodeBERT. Un modelo basado en *transformers* que se especializa en tareas relacionadas con la comprensión y generación de código.
- DeepCode. Un modelo de especializado en la revisión de código y detección de errores.

> Un proyecto que implica la generación automática de documentación a partir de código fuente podría usar GPT-3 para generar explicaciones detalladas en lenguaje natural basadas en fragmentos de código.

© Universidad Internacional de La Rioja (UNIR)

La Tabla 2 muestra una comparativa entre algunos de los principales modelos empleaos para la generación de código: ChatGPT, CodeBERT, Codex, BERT y DeepCode.

Parameter/ Model	ChatGPT (OpenAI)	CodeBERT (Microsoft)	Codex (OpenAI)	BERT	DeepCode
Primary use case	Conversational AI and code help	Code translation and embedding	Code generation and assistance	Code retrieval and semantic search	Bug detection and code review
Training data	Web text and code snippets	Parallel code in different languages	GitHub repositories	Code repositories	Large codebases
Model type	Transformer based	Transformer based	Transformer based	Transformer based	Neural based
Performance	High for general conversation	High for code translation	High for code suggestion	Moderate to high for code search	High for bug detection
Interactivity	Interactive	Mostly offline	Interactive	Interactive	Mostly offline

Tabla 2. Comparativa entre los principales modelos de generación de código. Fuente: Rahmaniar, 2024.

Integración del modelo en el flujo de trabajo

Una vez seleccionado el modelo, es crucial integrarlo en el flujo de trabajo de desarrollo de *software*. Los modelos generativos se pueden implementar de varias maneras, aunque depende de las necesidades del equipo de desarrollo:

- **Autocompletado inteligente.** El uso de autocompletado mediante modelos de lenguaje, como GitHub Copilot o IntelliCode, les permite a los desarrolladores escribir código más rápido y con menos errores. Estos modelos predicen la siguiente línea o bloque de código basándose en el contexto.

  - Por ejemplo, mientras que el desarrollador escribe, el modelo sugiere fragmentos de código en función de lo que se ha escrito. Esto acelera el proceso de codificación y reduce los errores humanos.

- **Generación de código a partir de descripciones.** Modelos como Codex les permiten a los desarrolladores escribir descripciones en lenguaje natural, que luego se traducen en código funcional. Esto es útil para generar rápidamente prototipos o funciones repetitivas.

© Universidad Internacional de La Rioja (UNIR)

- Por ejemplo, un desarrollador puede escribir «crear una función en Python que calcule el factorial de un número» y el modelo generaría automáticamente el código correspondiente.

- **Traducción de código entre lenguajes.** Los modelos generativos pueden traducir código de un lenguaje a otro, lo que facilita la interoperabilidad entre proyectos con múltiples lenguajes de programación.

- Por ejemplo, un equipo de desarrollo que trabaja con varias tecnologías (por ejemplo, *frontend* en JavaScript y *backend* en Python) puede utilizar un modelo generativo para traducir funciones clave entre ambos lenguajes de manera eficiente.

- **Generación automática de pruebas.** Los modelos pueden generar automáticamente pruebas unitarias o de integración para garantizar que el código funcione como se espera. Esto ahorra tiempo y esfuerzo al desarrollador.

- Por ejemplo, un modelo generativo puede analizar una función y generar automáticamente un conjunto de pruebas unitarias basadas en los casos de uso esperados.

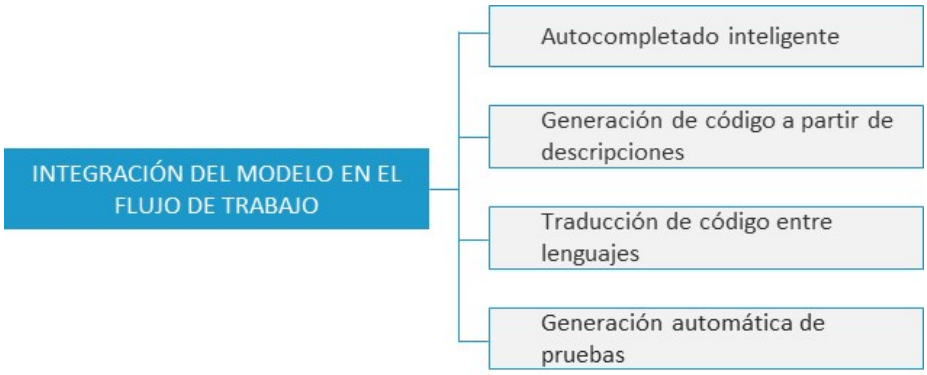


Figura 8. Integración del modelo en el flujo de trabajo. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Ajuste fino y personalización del modelo

Una vez que el modelo está integrado en el flujo de trabajo, se puede realizar un **ajuste fino** para adaptarlo mejor a las necesidades del proyecto específico. Este proceso implica **entrenar al modelo** con datos específicos del proyecto o **ajustar sus hiperparámetros** para mejorar el rendimiento.

- **Entrenamiento con datos propios.** Los modelos generativos preentrenados pueden ajustarse para generar códigos con características específicas de un proyecto en particular. Por ejemplo, si un equipo trabaja con un conjunto de bibliotecas o convenciones de codificación particulares, el modelo puede ser entrenado para generar código que siga esas reglas.

  - Por ejemplo, un equipo que usa una arquitectura orientada a microservicios puede entrenar un modelo para generar código con patrones y estructuras específicos de microservicios.

- **Personalización de sugerencias de código.** En herramientas de autocompletado, se puede personalizar el modelo para que proporcione sugerencias de código que se alineen con las convenciones de codificación del equipo o empresa, para mejorar así la consistencia y la calidad del código generado.

Herramientas para implementación práctica

Existen varias herramientas y plataformas que facilitan la implementación de modelos generativos en proyectos de desarrollo de *software:*

- GitHub Copilot. Herramienta de asistencia de programación que utiliza Codex para sugerir y completar fragmentos de código en varios lenguajes.

- Tabnine. Un asistente de código impulsado por IA que proporciona autocompletado avanzado y sugerencias basadas en el contexto.

© Universidad Internacional de La Rioja (UNIR)

- DeepCode. Un sistema basado en IA que analiza el código fuente para identificar errores, vulnerabilidades de seguridad y problemas de rendimiento, además de que proporciona sugerencias en tiempo real.
- Kite. Un asistente de código que utiliza IA para autocompletar líneas de código, mejorar la navegación y proporcionar documentación.

Evaluación del rendimiento del modelo

Evaluar el rendimiento de un modelo generativo es un paso crucial para asegurarse de que su implementación sea exitosa. Esto implica medir varios factores, como la precisión del código generado, su eficiencia y su capacidad para seguir las mejores prácticas.

- Métricas clave:
  - Precisión sintáctica y semántica: es importante que el código generado no solo sea sintácticamente correcto, sino también semánticamente adecuado para el problema en cuestión.
  - Eficiencia: se debe evaluar si el código generado es eficiente en términos de recursos y tiempo de ejecución.
  - Legibilidad: el código generado debe ser legible y fácil de entender, para que otros desarrolladores puedan trabajar con este sin dificultades.
- Retroalimentación del usuario. El *feedback* que los desarrolladores usan en el modelo y en su flujo de trabajo diario es invaluable para ajustar y mejorar el modelo. Las sugerencias que se ajustan a los patrones y estilos preferidos del equipo ayudan a mejorar la aceptación y la eficacia del modelo.

© Universidad Internacional de La Rioja (UNIR)

Casos de uso y ejemplos en la industria

- **Autocompletado en IDE (entornos de desarrollo integrados).** GitHub Copilot fue ampliamente adoptado por equipos de desarrollo para facilitar la escritura de código, especialmente en lenguajes como Python, JavaScript y Go. El modelo proporciona sugerencias contextuales basadas en el código que el desarrollador está escribiendo, por lo que acelera la creación de código repetitivo y estructurado.

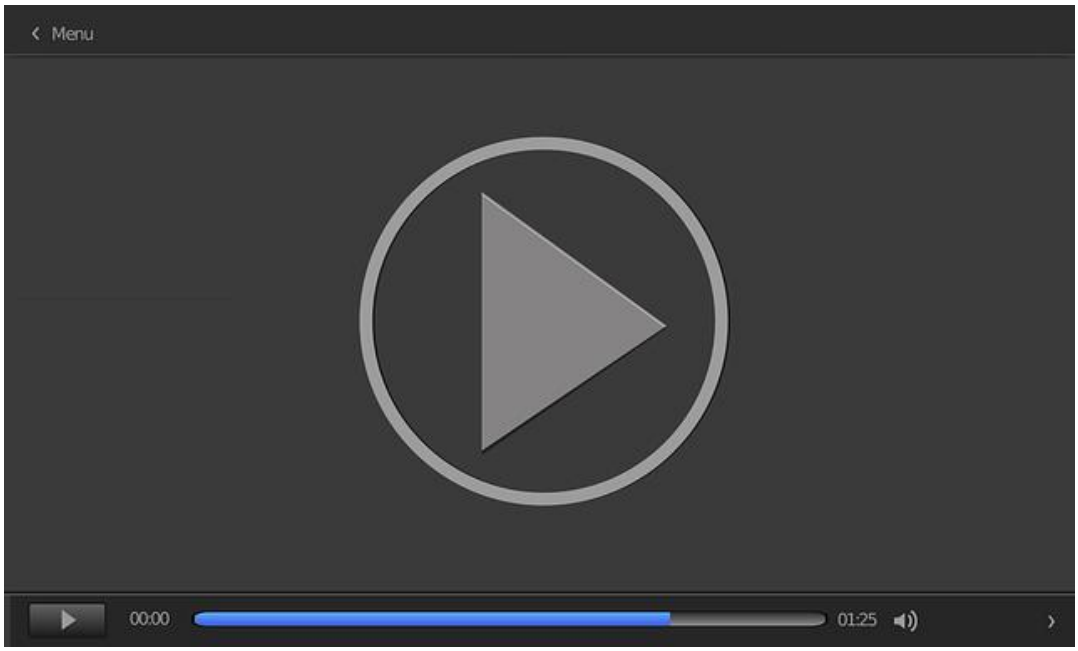
- **Automatización de pruebas en proyectos a gran escala.** Empresas como Microsoft y Facebook comenzaron a utilizar modelos generativos para generar automáticamente pruebas unitarias y de integración para grandes bases de código. Esto redujo el tiempo necesario para validar nuevas funciones y mejoró la cobertura de pruebas.

- **Generación de código para API específicas.** Algunas *startups* están utilizando modelos generativos ajustados a sus API para generar un código que interactúe correctamente con sus sistemas. Por ejemplo, se puede generar código para interactuar con una API de pago o un servicio de almacenamiento en la nube de manera automática.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

A continuación, veremos el vídeo *Principales avances en modelos generativos para Programación.* Este vídeo explorará los desarrollos más recientes en modelos de lenguaje específicos para programación, con énfasis en el análisis y evaluación de modelos avanzados como OpenAI Codex y AlphaCode. Se abordarán sus capacidades para resolver problemas complejos, escribir código eficiente y sus limitaciones actuales. También se discutirá la evolución de los modelos a partir de GPT-3 y cómo las nuevas arquitecturas están mejorando la precisión en la generación de código.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=2fb14730-8b97-42ba-a113-b21400e089fa

© Universidad Internacional de La Rioja (UNIR)

2.7. Referencias bibliográficas

Huan, J. I. N. y Qinying, L. I. (2023). Fine-Tuning Pre-Trained CodeBERT for Code Search in Smart Contract. *Wuhan University Journal of Natural Sciences, 28*(3), pp. 237-245.

Mohsin, A., Janicke, H., Wood, A., Sarker, I. H., Maglaras, L. y Janjua, N. (2024). Can We Trust Large Language Models Generated Code? A Framework for In-Context Learning, Security Patterns, and Code Evaluations Across Diverse LLMs. *ArXiv.* https://arxiv.org/abs/2406.12513

Rahmaniar, W. (2024). Chatgpt for software development: Opportunities and challenges. *IT Professional, 26*(3), pp. 80-86.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., Polosukhin, I. (2017). *Attention is all you need. Advances in Neural Information Processing Systems.* Curran Associates, Inc.

Visual Studio Code. (s. f.). *Code completions with GitHub Copilot in VS Code.* https://code.visualstudio.com/docs/copilot/ai-powered-suggestions

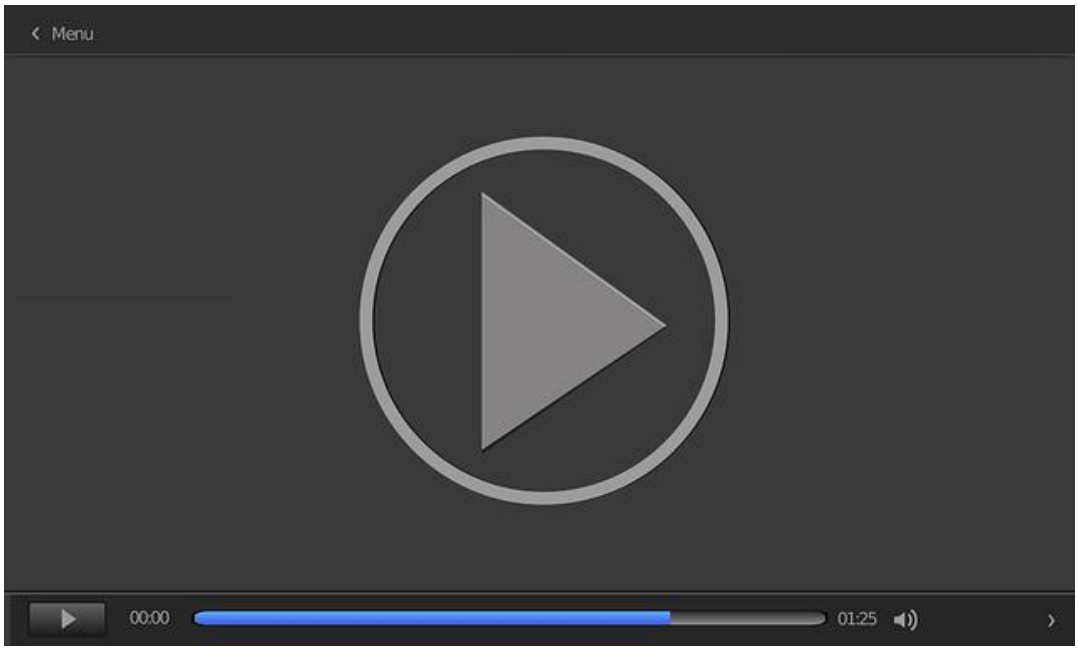
Zhu, Y., Zhang, Y., Yang, H. y Wang, F. (2019). *GANCoder: an automatic natural language-to-programming language translation approach based on GAN* [presentación en conferencia]. Natural Language Processing and Chinese Computing, Dunhuang, China. https://link.springer.com/chapter/10.1007/978-3-030-32236-6_48#:~:text=Abstract,the%20given%20natural%20language%20utterances.

© Universidad Internacional de La Rioja (UNIR)

OpenAI Codex: IA que escribe software

Lex Clips. (2021, septiembre 5). *OpenAI Codex: AI that writes software | Wojciech Zaremba and Lex Fridman* [Vídeo]. YouTube. https://youtu.be/omw1iWOSX-c?si=2JME3ZDsKz8ZDdiG

En esta entrevista, Wojciech Zaremba, cofundador de OpenAI, detalla cómo OpenAI Codex transforma la programación asistida por IA. Expone ejemplos reales de cómo los desarrolladores pueden aprovechar esta herramienta para escribir código más rápido y eficiente.



Accede al vídeo:

https://www.youtube.com/embed/omw1iWOSX-c?si=oCKGC5Ait8VJSozo

© Universidad Internacional de La Rioja (UNIR)

Exploring the limits of transfer learning with a unified text-to-text transformer

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W. y y Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research, 21*(140), pp. 1-67. https://jmlr.org/papers/v21/20-074.html

Este artículo explora los límites del aprendizaje por transferencia en modelos transformadores, por lo que presenta un enfoque *text-to-text* que es crucial para la generación automática de código. Es ideal para entender cómo funcionan los transformadores y su aplicabilidad en múltiples tareas, incluyendo programación.

© Universidad Internacional de La Rioja (UNIR)

Microsoft CodeBERT

Celbree. (s. f.). *CodeBERT.* GitHub. https://github.com/microsoft/CodeBERT

En este repositorio, se proporciona el modelo CodeBERT, que se preentrena con distintos modelos. Es una excelente referencia para quienes buscan utilizar IA en la optimización y generación de código más avanzado.

© Universidad Internacional de La Rioja (UNIR)

Test

1. ¿Qué es un modelo generativo en el contexto de la programación?

   A. Un modelo que predice la categoría de un dato.

   B. Un modelo que genera nuevos datos similares a los del conjunto de entrenamiento.

   C. Un modelo que clasifica imágenes.

   D. Un modelo que optimiza el rendimiento de las redes neuronales.

2. ¿Cuál es una de las principales aplicaciones de Codex?

   A. Traducción automática de textos.

   B. Generación de código a partir de descripciones en lenguaje natural.

   C. Clasificación de código en categorías.

   D. Detección de errores de memoria.

3. ¿Qué permite hacer la tecnología GitHub Copilot?

   A. Generar gráficos.

   B. Sugerir código en tiempo real con base en el contexto.

   C. Eliminar código redundante automáticamente.

   D. Detectar vulnerabilidades en tiempo real.

4. ¿Qué tipo de redes neuronales utilizan los *transformers*?

   A. Redes recurrentes.

   B. Redes convolucionales.

   C. Redes de memoria a corto plazo.

   D. Redes de atención autoreguladora.

© Universidad Internacional de La Rioja (UNIR)

5. ¿Cuál es una de las principales ventajas de los *transformers* frente a otros modelos en programación?

A. Pueden generar código sin errores.

B. Pueden procesar secuencias largas de datos de forma eficiente.

C. Son más rápidos que las redes convolucionales.

D. Pueden generar gráficos automáticamente.

6. ¿Qué significa la técnica de aprendizaje supervisado en el entrenamiento de modelos generativos?

A. El modelo se entrena sin etiquetar los datos.

B. El modelo predice las secuencias de código con base en datos etiquetados.

C. El modelo ajusta sus parámetros sin intervención humana.

D. El modelo genera código basado en datos no estructurados.

7. ¿Qué tipo de datos son ideales para entrenar modelos generativos de código?

A. Datos de audio.

B. Imágenes.

C. Código fuente en repositorios públicos.

D. Gráficos vectoriales.

8. ¿Qué hacen las GAN en la generación de código?

A. Compiten entre dos redes para mejorar la generación de datos.

B. Generan gráficos automáticamente.

C. Procesan grandes volúmenes de datos sin supervisión.

D. Optimizan el rendimiento de la memoria.

© Universidad Internacional de La Rioja (UNIR)

9. ¿Qué representa el discriminador en una GAN?

A. Un modelo que genera datos nuevos.

B. Un modelo que clasifica texto.

C. Un modelo que evalúa si los datos generados son auténticos.

D. Un modelo que predice las categorías de código.

10. ¿Cuál es uno de los principales desafíos de los modelos generativos en programación?

A. Producir código rápido.

B. Garantizar la seguridad y la eficiencia del código generado.

C. Predecir datos no secuenciales.

D. Traducir gráficos en tiempo real.

11. ¿Qué tipo de tareas puede realizar un modelo *transformer* entrenado en programación?

A. Detección de errores lógicos.

B. Generación automática de música.

C. Traducción de gráficos vectoriales.

D. Procesamiento de imágenes en 3D.

12. ¿Qué ventaja ofrecen las GAN en la optimización de código?

A. Generan código más legible.

B. Producen versiones optimizadas de código.

C. Traducen código automáticamente.

D. Eliminar comentarios innecesarios.

© Universidad Internacional de La Rioja (UNIR)

13. ¿Qué modelo es conocido por generar funciones completas en JavaScript?

A. CodeBERT.

B. Codex.

C. GraphCodeBERT.

D. GPT-2.

14. ¿Cómo se llaman los componentes que procesan las secuencias de entrada y salida en los *transformers*?

A. Neuronas.

B. Codificadores y decodificadores.

C. Redes convolucionales.

D. Redes recurrentes.

15. ¿Cuál de las siguientes tareas no es típica de un modelo generativo en programación?

A. Autocompletado.

B. Traducción de código.

C. Detección de virus.

D. Optimización de código.

16. ¿Qué es la *tokenización* en el preprocesamiento de datos de código?

A. Separar el código en bloques grandes.

B. Dividir el código en unidades mínimas de significado.

C. Normalizar las variables.

D. Ordenar el código alfabéticamente.

© Universidad Internacional de La Rioja (UNIR)

17. ¿Qué función tiene la sintonización fina de un modelo generativo?

A. Preentrenar el modelo con nuevos datos.

B. Ajustar un modelo preentrenado para tareas específicas.

C. Evaluar la calidad de los datos.

D. Predecir la siguiente secuencia de código.

18. ¿Cuál es una limitación común en los modelos generativos de código?

A. No pueden procesar grandes volúmenes de datos.

B. Solo funcionan con lenguajes de alto nivel.

C. Pueden generar soluciones repetitivas o subóptimas.

D. Solo generan código en Python.

19. ¿Qué es la regularización en el entrenamiento de un modelo?

A. Un proceso que elimina datos innecesarios.

B. Una técnica para evitar el sobreajuste en el modelo.

C. Un tipo de codificación de datos.

D. Un método para mejorar la legibilidad del código.

20. ¿Qué es el aprendizaje transferido en la sintonización de modelos generativos?

A. La capacidad de generar código sin datos.

B. Ajustar un modelo preentrenado para una nueva tarea.

C. Crear nuevos datos a partir de una muestra pequeña.

D. Predecir datos sin intervención humana.

© Universidad Internacional de La Rioja (UNIR)